

# AWS Messaging Services – Amazon SQS & Amazon MQ

## 1. Introduction

In AWS, messaging services are used to **send messages between different parts of an application** so that systems are **loosely coupled, scalable, and fault-tolerant**.

AWS provides two main messaging services:

- **Amazon SQS (Simple Queue Service)** – Fully managed queue service
- **Amazon MQ** – Managed message broker for traditional applications

## Amazon SQS (Simple Queue Service)

### —>What is Amazon SQS?

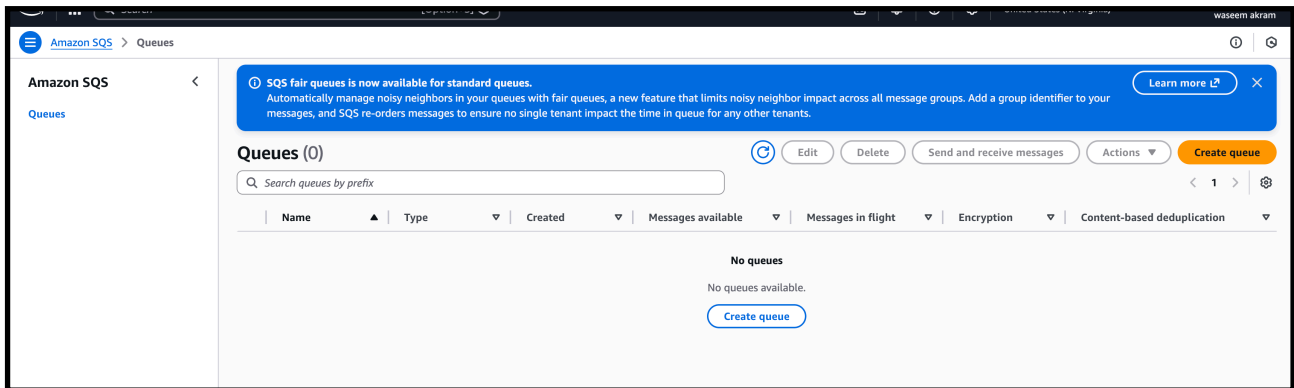
Amazon SQS is a **fully managed message queue service** that allows applications to **send, store, and receive messages** between components **without losing messages**.

- Messages are stored in a **queue**
- Producer sends message
- Consumer reads and deletes message

Task: Create an SQS Queue and Send & Receive Messages

## STEP 1: Open Amazon SQS

—> sqs —> queues —> create queues

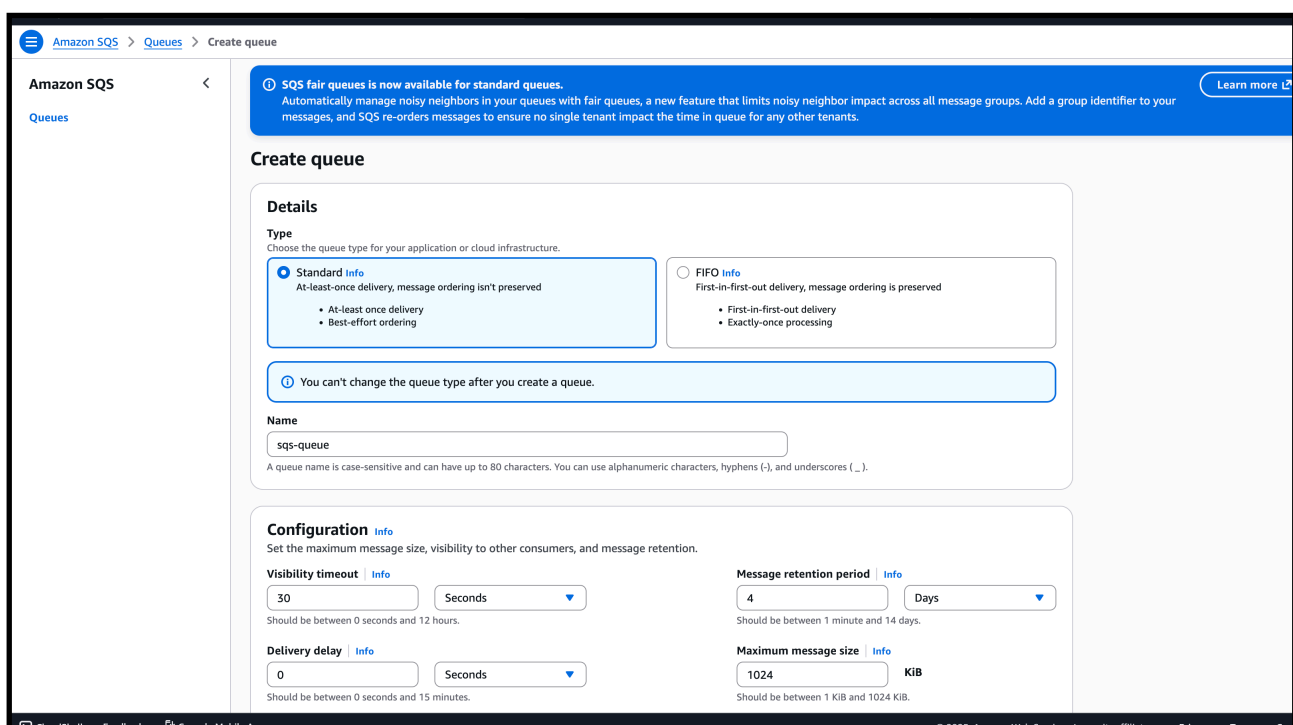


Opened Amazon SQS service from AWS Management Console.

## STEP 2: Create Queue

- Click **Create queue**
- Select **Standard** queue
- Enter queue name: **sqs-queue**

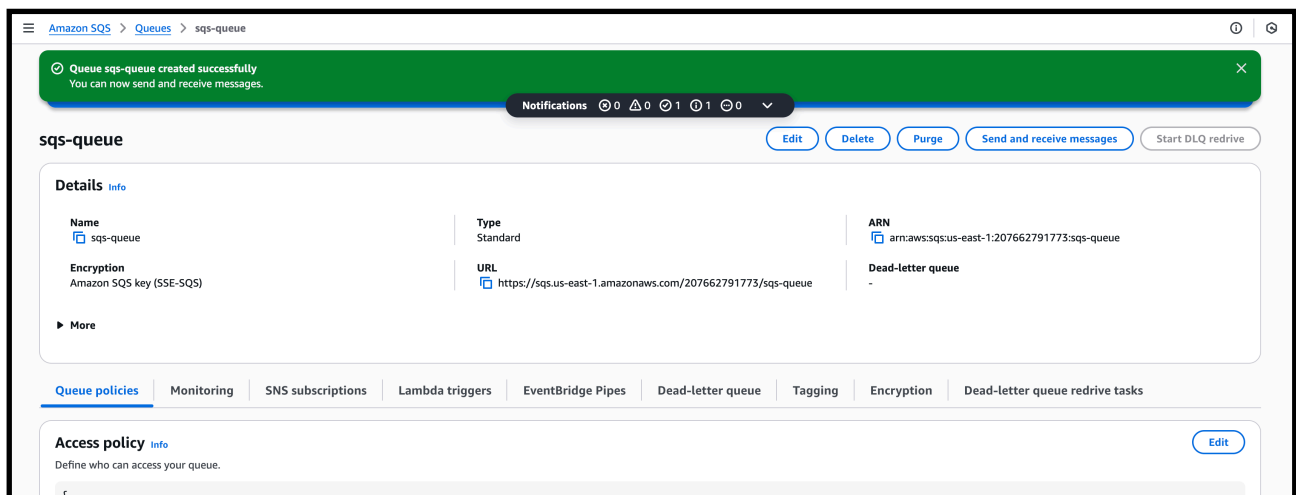
—> Configure Queue Settings



—> Created an Amazon SQS **Standard queue** named **sqs-queue**.

—> Configured SQS queue with default settings such as visibility timeout and message retention period.

### STEP 3: Queue Created Successfully



—> SQS queue **sqs-queue** created successfully.

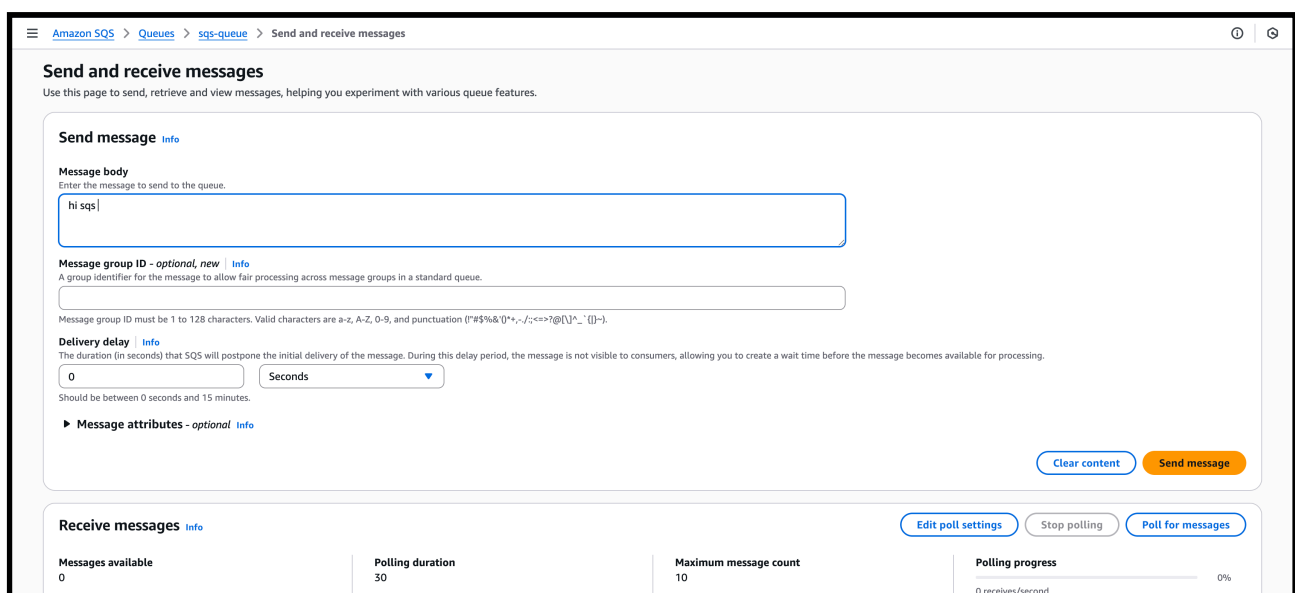
—> Queue creation success message appears

—> Queue details page opens automatically

## Open Send and Receive Messages Page

### Action

- Click **Send and receive messages**



—>Enter message body: `hi sqs`

Delivery delay: **0 seconds**

Click **Send message**

—> Sent a test message with body `hi sqs` to the SQS queue.

Next :

—> **Message Sent Successfully**

### Send and receive messages

Use this page to send, retrieve and view messages, helping you experiment with various queue features.

#### Send message [Info](#)

✔ Your message has been sent and is ready to be received.

[View sent message details](#) ×

**Message body**

Enter the message to send to the queue.

hi sqs

**Message group ID - optional, new** [Info](#)

A group identifier for the message to allow fair processing across message groups in a standard queue.

Message group ID must be 1 to 128 characters. Valid characters are a-z, A-Z, 0-9, and punctuation (!"#%&'()\*+,-./:;<=>?@[\\]^\_`{|}~).

**Delivery delay** [Info](#)

The duration (in seconds) that SQS will postpone the initial delivery of the message. During this delay period, the message is not visible to consumers, allowing you to create a wait time before the message becomes available for processing.

0

Seconds

Should be between 0 seconds and 15 minutes.

[▶ Message attributes - optional](#) [Info](#)

## Action

- Green success message appears
- Message ready to be received

—> **Receive Message from Queue**

Click **Poll for messages**

Message is received

**Message group ID - optional, new** [Info](#)

A group identifier for the message to allow fair processing across message groups in a standard queue.

Message group ID must be 1 to 128 characters. Valid characters are a-z, A-Z, 0-9, and punctuation (!"#%&'()\*+,-./:;<=>?@[\\]^\_`{|}~).

**Delivery delay** [Info](#)

The duration (in seconds) that SQS will postpone the initial delivery of the message. During this delay period, the message is not visible to consumers, allowing you to create a wait time before the message becomes available for processing.

0

Seconds

Should be between 0 seconds and 15 minutes.

[▶ Message attributes - optional](#) [Info](#)

#### Received message: 724563c4-7417-4560-917d-c927649f3837

Body

Attributes

Details

hi sqs

Done

**Receive messages** [Info](#)

Messages available

1

Polling duration

30

Maximum message count

10

Polling progress

0 receives/second

37%

View details

Delete

Messages (1)

Search messages

| <input type="checkbox"/> | ID                                   | Sent                   | Size    | Receive count |
|--------------------------|--------------------------------------|------------------------|---------|---------------|
| <input type="checkbox"/> | 724563c4-7417-4560-917d-c927649f3837 | 2025-12-19T16:19+05:30 | 7 bytes | 1             |

—> Polled the queue and received the message successfully.

## Architecture Flow (SQS)

Producer → SQS Queue → Consumer

1. Producer sends message to SQS
2. SQS stores message durably
3. Consumer polls the queue
4. Message processed and deleted

## Amazon MQ

Amazon MQ is a **managed message broker service** for applications that use traditional messaging protocols.

Supports:

- ActiveMQ
- RabbitMQ

## Use Cases

- Migrating legacy messaging systems
- Applications using JMS, AMQP
- Complex routing and pub/sub models

## Architecture Flow (Amazon MQ)

Producer → Amazon MQ Broker → Consumer

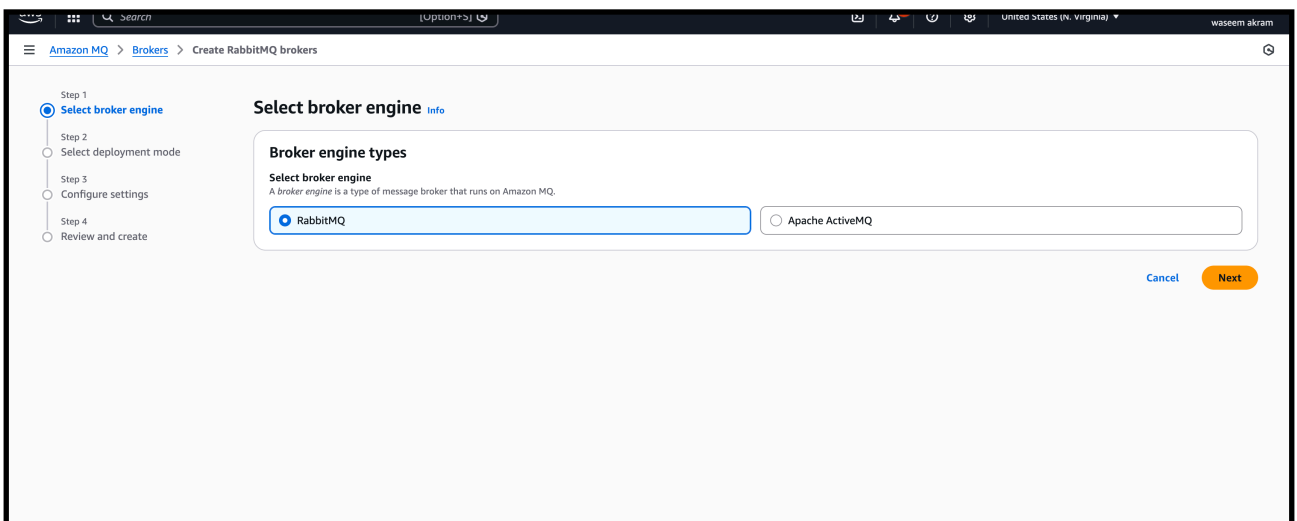
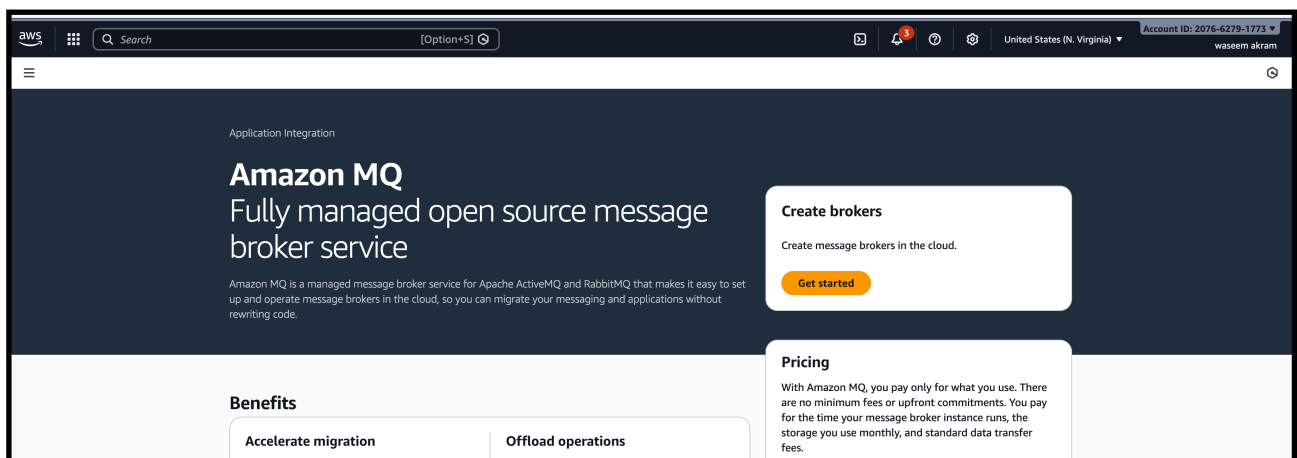
1. Producer connects to broker
2. Message published to queue/topic
3. Broker routes message
4. Consumer receives message

## Step-by-Step Execution (Amazon MQ)

### Step 1: Create Broker

**AWS Console Path:** Amazon MQ → Create broker

- Broker engine: ActiveMQ
- Deployment: Single-instance



—> deselect the deployment mode  
(Cluster deployment )

The screenshot shows the 'Select deployment mode' step in the Amazon MQ console. On the left, a progress bar indicates four steps: Step 1 (Select broker engine), Step 2 (Select deployment mode), Step 3 (Configure settings), and Step 4 (Review and create). Step 2 is currently selected. The main area is titled 'Select deployment mode' and contains two options: 'Single-instance broker' and 'Cluster deployment'. The 'Cluster deployment' option is selected with a blue radio button. Below each option is a diagram: a single circle for the single-instance broker and a triangle of three circles for the cluster deployment. At the bottom right, there are 'Cancel', 'Previous', and 'Next' buttons.

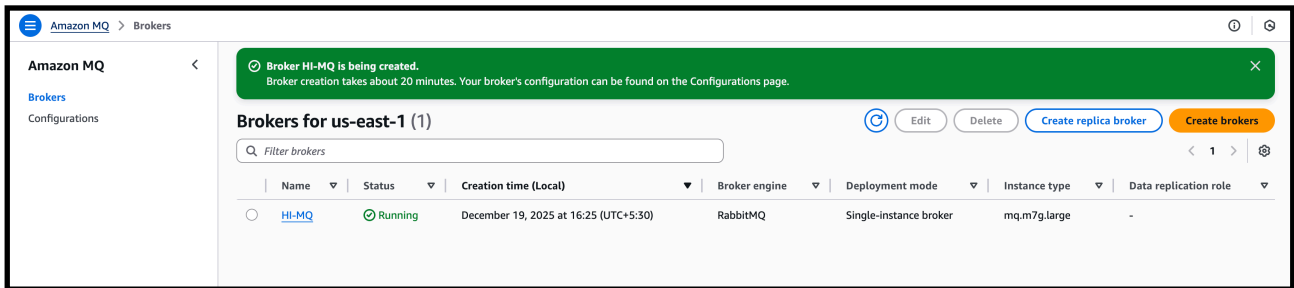
## Create Broker User

- Username: myuser3
- Password: waseem@20034a

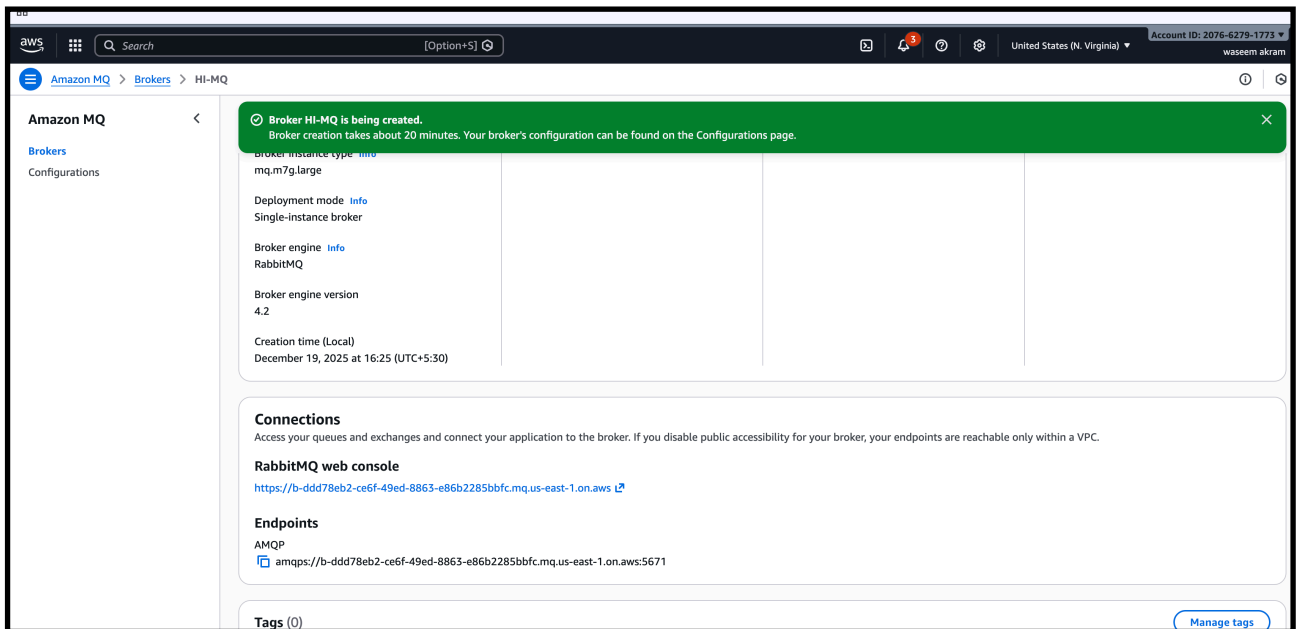
The screenshot shows the 'Configure settings' step in the Amazon MQ console. The progress bar on the left shows Step 3 (Configure settings) as the current step. The main area is titled 'Configure settings' and is divided into two sections: 'Details' and 'RabbitMQ access'. In the 'Details' section, there are fields for 'Broker name' (HI-MQ), 'Broker instance type' (mq.m7g.large), 'Deployment mode' (Single-instance broker), 'Estimated deployment time' (20 minutes), 'Storage type' (Amazon Elastic Block Store), and 'Broker engine' (RabbitMQ). In the 'RabbitMQ access' section, there are fields for 'Username' (myuser3) and 'Password' (Waseem@20034a). There is also a checkbox for 'Enable additional access methods' which is currently unchecked.



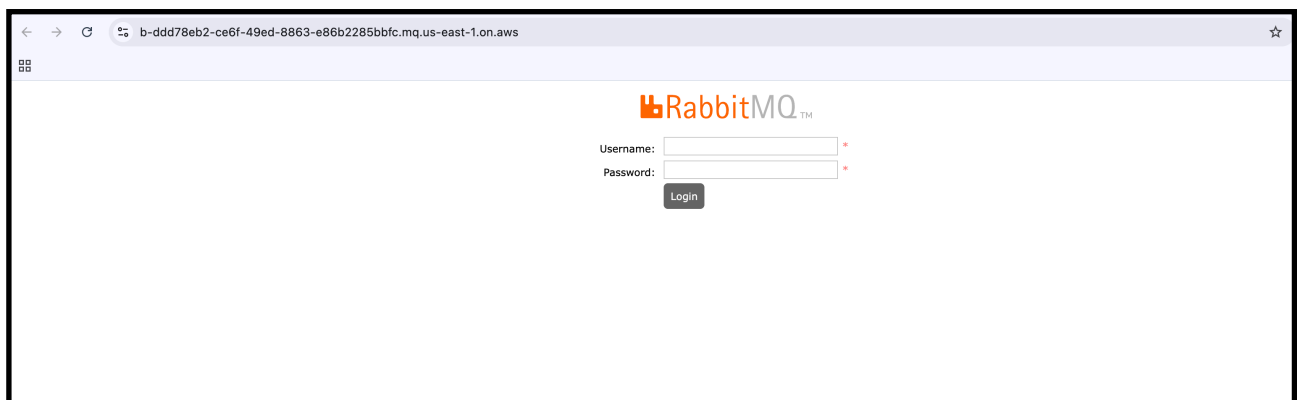
—> wait until the status is running



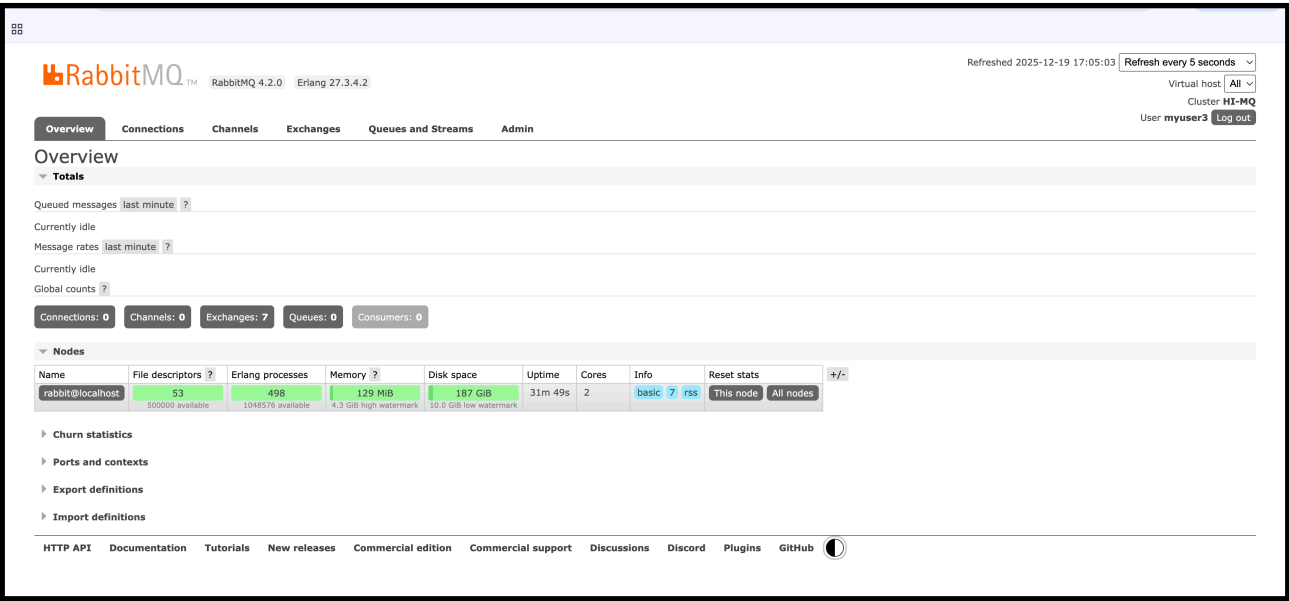
—> now go to the rabbit MQ web console and copy the url and search in browser



—> now the add the user name and password which we created in MQ



—> Connect Application



# SQS vs Amazon MQ (Comparison)

| Feature     | SQS               | Amazon MQ         |
|-------------|-------------------|-------------------|
| Type        | Serverless        | Broker-based      |
| Protocol    | AWS proprietary   | AMQP, JMS, MQTT   |
| Scalability | Automatic         | Manual            |
| Management  | Fully managed     | Partially managed |
| Use Case    | Cloud-native apps | Legacy apps       |

Conclusion:  
Amazon SQS and Amazon MQ together cover both **modern cloud-native** and **legacy messaging** requirements in AWS.